

UD1 - RA1: Modelos de Regresión con Python y Scikit-learn

Regresión Lineal, Árboles de Decisión y Random Forest

Módulo: Inteligencia Artificial y Computación en la Nube **Ciclo:** 2º DAM / DAW **RA1:** Desarrolla programas en lenguajes de programación enfocados a Inteligencia Artificial (IA), utilizando técnicas de procesamiento de datos.

Índice

1. [Introducción: ¿qué es un problema de regresión?](#)
 2. [Preparación del entorno y del dataset](#)
 3. [Regresión Lineal](#)
 4. [Árboles de Decisión \(Decision Tree\)](#)
 5. [Random Forest \(Bosques Aleatorios\)](#)
 6. [Métricas de evaluación](#)
 7. [Comparativa de los tres modelos](#)
 8. [Resumen final](#)
 9. [Ejercicios propuestos](#)
-

1. Introducción: ¿qué es un problema de regresión?

En Inteligencia Artificial, y más concretamente en **Aprendizaje Automático (Machine Learning)**, distinguimos dos grandes tipos de problemas de aprendizaje supervisado:

- **Clasificación:** el modelo predice una **categoría** (por ejemplo: “spam” / “no spam”, “aprobado” / “suspense”).
- **Regresión:** el modelo predice un **valor numérico continuo** (por ejemplo: el precio de una vivienda, la temperatura de mañana, el consumo de gasolina de un coche).

En este documento vamos a estudiar **tres algoritmos** muy utilizados para resolver problemas de regresión:

Algoritmo	Idea principal
Regresión Lineal	Ajusta una línea (o hiperplano) recta a los datos
Árbol de Decisión	Divide los datos en regiones mediante preguntas tipo “if/else”
Random Forest	Combina muchos árboles de decisión para mejorar la predicción

Todos ellos se implementarán con la librería **scikit-learn**, el estándar de facto en Python para Machine Learning clásico.

2. Preparación del entorno y del dataset

2.1 Instalación de librerías

```
pip install scikit-learn pandas matplotlib numpy
```

2.2 El dataset: California Housing

Scikit-learn incluye varios datasets “de juguete” listos para practicar, sin necesidad de descargar nada externo. Para regresión usaremos el dataset **California Housing**, que contiene información de distritos de California (ingresos medios, número de habitaciones, ubicación, etc.) y como variable objetivo el **precio medio de la vivienda**.

```
from sklearn.datasets import fetch_california_housing
import pandas as pd

# Cargamos el dataset
datos = fetch_california_housing(as_frame=True)

# X: variables predictoras (features)
X = datos.data

# y: variable objetivo (target) -> precio medio de la vivienda
y = datos.target

print(X.head())
print(y.head())
print(datos.DESCR) # Descripción completa del dataset
```

2.3 División en entrenamiento y prueba

Antes de entrenar cualquier modelo, siempre dividimos los datos en dos conjuntos:

- **Entrenamiento (train):** con el que el modelo “aprende”.
- **Prueba (test):** con el que comprobamos si el modelo generaliza bien a datos que nunca ha visto.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% de los datos para test
    random_state=42     # semilla fija, para que el resultado sea reproducible
)

print(f"Ejemplos de entrenamiento: {len(X_train)}")
print(f"Ejemplos de prueba: {len(X_test)}")
```

Este paso es común a los tres algoritmos que vamos a ver.

3. Regresión Lineal

3.1 ¿Qué es?

La **regresión lineal** es el modelo más sencillo e intuitivo de Machine Learning. Parte de la idea de que existe una **relación lineal** (proporcional) entre las variables de entrada (X) y la variable de salida (y).

En su versión más simple, con una sola variable, la regresión lineal busca la recta que mejor se ajusta a los puntos:

$$y = m \cdot x + b$$

Donde: - **m** es la pendiente de la recta (cuánto cambia y por cada unidad que cambia x) - **b** es el punto de corte con el eje Y (el valor de y cuando $x = 0$)

Cuando tenemos varias variables de entrada (lo habitual en problemas reales), la fórmula se generaliza:

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + \dots + w_n \cdot x_n + b$$

Cada variable x_i tiene asociado un **peso o coeficiente w_i** , que indica cuánto influye esa variable en el resultado final.

3.2 ¿Cómo “aprende” el modelo?

“Entrenar” una regresión lineal consiste en encontrar los valores de w_1 , w_2 , ..., w_n y b que hacen que las predicciones del modelo se acerquen lo máximo posible a los valores reales.

Para medir “lo máximo posible” se utiliza una **función de coste** (o de error), normalmente el **Error Cuadrático Medio (MSE)**:

$$MSE = (1/n) \cdot \sum (y_{\text{real}} - y_{\text{predicho}})^2$$

El algoritmo ajusta los pesos para que este error sea lo más pequeño posible (normalmente mediante un método matemático llamado **mínimos cuadrados**, aunque scikit-learn hace este cálculo internamente por nosotros).

3.3 Ventajas e inconvenientes

□ Ventajas	□ Inconvenientes
Muy fácil de entender e interpretar	Solo funciona bien si la relación entre datos es (aproximadamente) lineal
Rápido de entrenar, incluso con muchos datos	Sensible a valores atípicos (<i>outliers</i>)
Los coeficientes explican la influencia de cada variable	No capta relaciones complejas o no lineales

3.4 Implementación en Python

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Creamos el modelo
modelo_lineal = LinearRegression()

# 2. Entrenamos el modelo con los datos de entrenamiento
modelo_lineal.fit(X_train, y_train)

# 3. Hacemos predicciones sobre el conjunto de test
y_pred_lineal = modelo_lineal.predict(X_test)

# 4. Evaluamos el resultado
```

```

mse = mean_squared_error(y_test, y_pred_lineal)
r2 = r2_score(y_test, y_pred_lineal)

print(f"MSE: {mse:.3f}")
print(f"R²: {r2:.3f}")

# Consultamos los coeficientes que ha aprendido el modelo
for nombre, peso in zip(X.columns, modelo_lineal.coef_):
    print(f"{nombre}: {peso:.4f}")

print(f"Término independiente (b): {modelo_lineal.intercept_:.4f}")

```

Con solo 4 líneas de código (crear, fit, predict, evaluar) hemos entrenado nuestro primer modelo. Este patrón (**crear** → **entrenar** → **predecir** → **evaluar**) se repite en prácticamente todos los algoritmos de scikit-learn.

4. Árboles de Decisión (Decision Tree)

4.1 ¿Qué es?

Un **árbol de decisión** es un modelo que toma decisiones haciendo una serie de **preguntas tipo sí/no** sobre las variables de entrada, hasta llegar a una predicción final.

Piensa en el típico juego de “adivina el número” o en un diagrama de flujo:

```

¿Ingresos medios > 5?
├── SÍ → ¿Nº habitaciones > 6?
│   ├── SÍ → Precio estimado: 4.5
│   └── NO → Precio estimado: 3.2
└── NO → ¿Antigüedad > 30 años?
    ├── SÍ → Precio estimado: 1.1
    └── NO → Precio estimado: 1.8

```

Cada “pregunta” se llama **nodo**, cada rama es una posible respuesta, y las predicciones finales se encuentran en las **hojas** del árbol.

4.2 ¿Cómo se construye el árbol?

El algoritmo busca, en cada paso, la variable y el valor de corte que **mejor separan los datos**, es decir, que agrupan en cada rama observaciones con valores de y lo más parecidos posible entre sí (menor error).

Para problemas de regresión, en cada división se calcula cuánto se reduciría el error (normalmente el **MSE**) si se hiciera esa pregunta. El árbol elige siempre la pregunta que más reduce el error, y repite el proceso de forma recursiva.

Este proceso continúa hasta que: - Se alcanza una **profundidad máxima** definida por nosotros, o - Un nodo no se puede dividir más (queda muy poca información), o - El error ya no mejora de forma significativa.

4.3 El problema del sobreajuste (overfitting)

Si dejamos que el árbol crezca sin límite, puede acabar memorizando los datos de entrenamiento (incluido el ruido), en lugar de aprender un patrón general. Esto se llama **sobreajuste (overfitting)**: el modelo funciona genial en train, pero mal en test.

Para evitarlo, controlamos parámetros como: - `max_depth`: profundidad máxima del árbol. - `min_samples_leaf`: número mínimo de observaciones que debe tener una hoja. - `min_samples_split`: número mínimo de observaciones necesarias para dividir un nodo.

4.4 Ventajas e inconvenientes

□ Ventajas	□ Inconvenientes
Fácil de visualizar e interpretar Captura relaciones no lineales	Tiende al sobreajuste si no se controla su tamaño Poco estable: pequeños cambios en los datos pueden generar árboles muy distintos
No requiere normalizar los datos	Por sí solo, suele predecir peor que modelos más avanzados

4.5 Implementación en Python

```
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt

# 1. Creamos el modelo, limitando su profundidad para evitar overfitting
modelo_arbol = DecisionTreeRegressor(
    max_depth=5,
    min_samples_leaf=10,
    random_state=42
)

# 2. Entrenamos
modelo_arbol.fit(X_train, y_train)

# 3. Predecimos
y_pred_arbol = modelo_arbol.predict(X_test)

# 4. Evaluamos
mse = mean_squared_error(y_test, y_pred_arbol)
r2 = r2_score(y_test, y_pred_arbol)

print(f"MSE: {mse:.3f}")
print(f"R²: {r2:.3f}")

# Visualizamos el árbol (útil para explicar el modelo)
plt.figure(figsize=(20, 10))
plot_tree(
    modelo_arbol,
    feature_names=X.columns,
    filled=True,
    max_depth=2, # mostramos solo los primeros niveles para que sea legible
    fontsize=10
)
plt.show()
```

5. Random Forest (Bosques Aleatorios)

5.1 ¿Qué es?

Un solo árbol de decisión puede ser inestable o poco preciso. La solución: **en lugar de usar un único árbol, entrenamos muchos árboles distintos y combinamos sus predicciones**. A esta técnica se le llama **ensemble learning** (aprendizaje en conjunto), y el Random Forest es uno de sus ejemplos más populares.

La idea, resumida:

“La sabiduría de la multitud”: si preguntamos a 200 expertos (árboles) y promediamos sus respuestas, el resultado suele ser más fiable que preguntar a un único experto.

5.2 ¿Cómo funciona internamente?

Random Forest combina dos ideas clave para conseguir que los árboles sean **diferentes entre sí** (si todos los árboles fueran iguales, promediarlos no aportaría nada):

1. **Bagging (Bootstrap Aggregating)**: cada árbol se entrena con una muestra aleatoria de los datos de entrenamiento, extraída **con reemplazo** (algunos ejemplos se repiten, otros no aparecen).
2. **Selección aleatoria de variables**: en cada división de cada árbol, solo se considera un subconjunto aleatorio de las variables disponibles (no todas). Esto obliga a los árboles a “explorar” diferentes patrones en los datos.

Una vez entrenados todos los árboles (por ejemplo, 100 o 200), la predicción final para regresión se calcula como el **promedio** de las predicciones de todos ellos:

$$\text{predicción_final} = (\text{predicción_árbol}_1 + \text{predicción_árbol}_2 + \dots + \text{predicción_árbol}_n) / n$$

5.3 Ventajas e inconvenientes

□ Ventajas	□ Inconvenientes
Mucho más preciso y estable que un único árbol	Más lento de entrenar (hay que construir muchos árboles)
Reduce el sobreajuste frente a un árbol individual	Más difícil de interpretar y visualizar (“caja negra”)
Funciona bien “out of the box”, sin apenas ajustar parámetros	Ocupa más memoria (hay que guardar todos los árboles)

5.4 Implementación en Python

```
from sklearn.ensemble import RandomForestRegressor

# 1. Creamos el modelo
modelo_bosque = RandomForestRegressor(
    n_estimators=200,      # número de árboles del bosque
    max_depth=10,         # profundidad máxima de cada árbol
    random_state=42,
    n_jobs=-1              # usa todos los núcleos de CPU disponibles
)

# 2. Entrenamos
modelo_bosque.fit(X_train, y_train)

# 3. Predecimos
y_pred_bosque = modelo_bosque.predict(X_test)

# 4. Evaluamos
mse = mean_squared_error(y_test, y_pred_bosque)
r2 = r2_score(y_test, y_pred_bosque)

print(f"MSE: {mse:.3f}")
print(f"R²: {r2:.3f}")

# Bonus: importancia de cada variable según el modelo
importancias = pd.Series(modelo_bosque.feature_importances_, index=X.columns)
print(importancias.sort_values(ascending=False))
```

El atributo `feature_importances_` es muy útil en la práctica profesional: nos indica **qué variables han sido más relevantes** para las predicciones del modelo.

6. Métricas de evaluación

Para saber si un modelo de regresión es bueno, no basta con “mirarlo por encima”; usamos métricas numéricas. Las más comunes:

Métrica	Fórmula (idea)	Interpretación
MAE (Error Absoluto Medio)	Media de $ y_{\text{real}} - y_{\text{predicho}} $	Error medio, en las mismas unidades que y . Fácil de interpretar.
MSE (Error Cuadrático Medio)	Media de $(y_{\text{real}} - y_{\text{predicho}})^2$	Penaliza más los errores grandes. Muy usado como función de coste.
RMSE (Raíz del MSE)	$\sqrt{\text{MSE}}$	Igual que el MSE pero en las unidades originales de y .
R² (Coeficiente de determinación)	Entre 0 y 1 (puede ser negativo)	Indica qué proporción de la variabilidad de y explica el modelo. Cuanto más cerca de 1, mejor.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred_bosque)
mse = mean_squared_error(y_test, y_pred_bosque)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred_bosque)

print(f"MAE: {mae:.3f}")
print(f"MSE: {mse:.3f}")
print(f"RMSE: {rmse:.3f}")
print(f"R²: {r2:.3f}")
```

□ Un **R² = 1** significa que el modelo predice perfectamente. Un **R² = 0** significa que el modelo no es mejor que predecir siempre la media de y . Un **R² negativo** indica que el modelo es peor que esa media.

7. Comparativa de los tres modelos

```
resultados = pd.DataFrame({
    "Modelo": ["Regresión Lineal", "Árbol de Decisión", "Random Forest"],
    "MSE": [
        mean_squared_error(y_test, y_pred_lineal),
        mean_squared_error(y_test, y_pred_arbol),
        mean_squared_error(y_test, y_pred_bosque)
    ],
    "R2": [
        r2_score(y_test, y_pred_lineal),
        r2_score(y_test, y_pred_arbol),
        r2_score(y_test, y_pred_bosque)
    ]
})
```

```
]
})

print(resultados)
```

Aspecto	Regresión Lineal	Árbol de Decisión	Random Forest
Tipo de relación que capta	Lineal	No lineal	No lineal
Interpretabilidad	Muy alta	Alta	Baja
Riesgo de overfitting	Bajo	Alto	Bajo-medio
Precisión típica	Media	Media	Alta
Velocidad de entrenamiento	Muy rápida	Rápida	Más lenta
Sensible a outliers	Sí	Poco	Poco

Conclusión práctica: en la mayoría de problemas reales, **Random Forest suele obtener mejores resultados** que un único árbol o que la regresión lineal, a costa de ser más difícil de interpretar. La regresión lineal sigue siendo muy útil como primer modelo de referencia (*baseline*) y cuando necesitamos explicar claramente el resultado.

8. Resumen final

- Un problema de **regresión** consiste en predecir un valor numérico continuo.
- La **regresión lineal** ajusta una relación lineal entre las variables de entrada y la salida, minimizando el error cuadrático medio.
- Un **árbol de decisión** divide los datos mediante preguntas sucesivas, formando regiones cada vez más homogéneas respecto a la variable objetivo.
- **Random Forest** combina muchos árboles de decisión entrenados con datos y variables ligeramente distintos (bagging), promediando sus predicciones para obtener un resultado más robusto y preciso.
- Con scikit-learn, el flujo de trabajo es siempre el mismo: **cargar datos** → **dividir en train/test** → **crear modelo** → **fit()** → **predict()** → **evaluar con métricas**.

9. Ejercicios propuestos

1. Carga el dataset `load_diabetes()` de scikit-learn y repite los tres modelos (regresión lineal, árbol de decisión y random forest). ¿Cuál obtiene mejor R^2 ?
2. Cambia el parámetro `max_depth` del árbol de decisión a los valores 2, 5, 10 y None (sin límite). Compara el MSE en train y en test para cada caso y explica qué ocurre con el overfitting.
3. Cambia el número de árboles (`n_estimators`) del Random Forest a 10, 50, 200 y 500. ¿A partir de qué valor deja de mejorar significativamente el resultado?
4. Usando `feature_importances_` del Random Forest, indica cuáles son las 3 variables más importantes del dataset California Housing para predecir el precio de la vivienda.
5. Dibuja con matplotlib un gráfico de dispersión comparando `y_test` (valores reales) frente a las predicciones de cada uno de los tres modelos. ¿Qué modelo se ajusta mejor a la diagonal ideal (predicción = valor real)?

Documento elaborado como material de apoyo teórico-práctico para el módulo de Inteligencia Artificial y Computación en la Nube — 2º DAM/DAW.